

Codeforces Round 1093 (Div. 2)

A. Blocked

1 second, 256 megabytes

Given an array a of integers of size n , we say that a position $1 \leq i \leq n$ is *blocked* if a_i can be expressed as the sum of a subset of $a_1, a_2, \dots, a_{i-1}$ (i.e. there exist $1 \leq j_1 < j_2 < \dots < j_k \leq i - 1$ such that $a_{j_1} + a_{j_2} + \dots + a_{j_k} = a_i$). Reorder a so that no position is blocked or report that it is impossible.

For example, reordering the array $[3, 2, 5]$ to $[2, 3, 5]$ makes position 3 blocked, since we can express $5 = 2 + 3$, but if we reorder it to $[3, 5, 2]$, no position is blocked.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 400$). The description of the test cases follows.

The first line of each test case contains an integer n ($1 \leq n \leq 200$).

The second line contains n integers, denoting the array a ($1 \leq a_i \leq 100$).

Output

For each test case, print any order of a such that no position is blocked if it exists, otherwise print -1 .

input
4
3
1 5 9
4
1 3 3 2
3
1 2 3
1
1
output
5 9 1
-1
3 1 2
1

In the third test case, the array $[3, 1, 2]$ has no position blocked:

Position 1 is not blocked since 3 can't be expressed as the sum of a subset of $[]$.

Position 2 is not blocked since 1 can't be expressed as the sum of a subset of $[3]$.

Position 3 is not blocked since 2 can't be expressed as the sum of a subset of $[3, 1]$.

B. OIE Excursion

1 second, 256 megabytes

Hector is on an excursion with the Spanish Olympiad in Informatics, visiting A Coruña, but he is desperate to sneak away to meet his friends Gustavo, Esomer, and Dani. To do so, he must cross a path guarded by n volunteers standing in a row, numbered from 1 to n ; the i -th volunteer watches over position i .

Each volunteer has an internal timer. Initially (at second 0), the i -th volunteer's timer has a value a_i . Every second, all timers increase by 1. Once a timer reaches m , it wraps around to 0. Specifically, at second x , the i -th volunteer's timer shows the value $(a_i + x) \pmod m$.

The i -th volunteer is only watching over position i when their timer is exactly 0. At any other time, they are distracted and will not notice Hector.

Hector starts at position 0, to the left of all volunteers. To escape, he must reach position $n + 1$. In the **end** of each second, Hector can choose to stay in his current position, move one position to the left, or move one position to the right. Note that Hector can't go to the left of position 0.

Hector is caught if and only if, at the **beginning** of a second, he is at position i ($1 \leq i \leq n$) and the i -th volunteer's timer is 0.

Determine whether there exists a strategy for Hector to pass all volunteers and escape without being caught.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

Each test case consists of two lines:

- The first line contains two integers n and m ($2 \leq n \leq 2 \cdot 10^5$, $2 \leq m \leq 10^9$) — the number of volunteers and the cycle length of the timers.
- The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < m$), where a_i represents the initial value of the i -th volunteer's timer at $t = 0$.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a line containing either "YES" or "NO", representing whether Hector can escape. You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

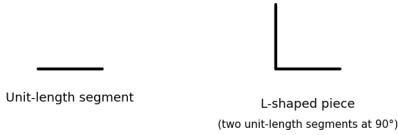
input
6
8 5
0 4 0 2 1 0 0 3
6 2
1 0 1 0 1 0
6 2
1 1 1 1 0 1
2 10
6 9
2 2
0 1
5 1000000000
1 2 3 4 999999999
output
YES
YES
NO
YES
YES
YES

- Case 1: Hector can move right every second without waiting or moving left and escape, because there is no i such that $(a_i + i) \pmod m = 0$.
- Case 2: One possible strategy is to wait for one second at the starting position and then move right every second without further waiting or moving left.

C. Grid L

2 seconds, 256 megabytes

Roger has p unit-length segments and q L-shaped pieces, each formed by joining two unit-length segments at a right angle.



He wants to use all of these pieces, with no piece left unused, to form a grid of dimensions $n \times m$.

Given p and q , determine whether there exist positive integers n and m such that an $n \times m$ grid can be formed using exactly p unit-length segments and q L-shaped pieces, possibly rotating them, and provide any valid n and m .

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 100$). The description of the test cases follows.

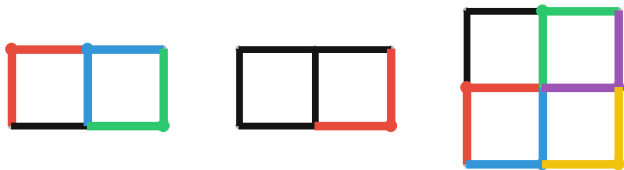
The first line of each test case contains two integers p and q ($1 \leq p, q \leq 10^8$).

Output

For each test case, print any valid n and m such that it's possible to construct $n \times m$ using all pieces. If there are no such n and m , print -1 .

input
7
1 2
1 3
5 1
2 5
2 10
100000000 100000000
1 1
output
-1
1 2
1 2
2 2
2 4
-1
-1

Here are the constructions for examples 2, 3, and 4. We use a different color for different L-shaped pieces, black for all unit-length segments.



D1. Unique Values (Easy version)

2 seconds, 256 megabytes

The difference between the easy version and hard version is the maximum number of queries allowed. In this version, it is 66.

There is a secret array a of length $2n + 1$, whose elements are integers from 1 to n . Each value appears exactly twice, except for one value, which appears exactly three times.

Your goal is to find the three positions of the value that appears three times.

To do this, you may ask the following query at most 66 times:

- Choose an integer k and an array s of k distinct indices between 1 and $2n + 1$.
- You will receive the number of different values among $a_{s_1}, a_{s_2}, \dots, a_{s_k}$ that appear exactly once, or, in other words, the

Problems - Codeforces

count of values that are not repeated.

For example, if the values $a_{s_1}, \dots, a_{s_k}$ are $\{2, 1, 2, 3, 2, 3, 6, 7\}$, the answer to the query will be 3, as 1, 6, 7 are the only values that appear exactly once. 3 appears 2 times and 2 appears 3 times, which is more than once, so they are not counted.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 500$). The description of the test cases follows.

The first line of each test case contains a single integer n ($2 \leq n \leq 1000$).

The array a is fixed for the test case and does not change during the interaction. In other words, the interactor is **not adaptive**.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^4$.

Interaction

For each test case, first read a single integer n .

You may ask up to 66 queries in each test case.

To ask a query, print a line in the format:

- $? k s_1 s_2 \dots s_k$ ($s_i \neq s_j$ for $i \neq j$, $1 \leq s_i \leq 2 \cdot n + 1$)

As a response to the query, you will receive the number of different values among $a_{s_1}, a_{s_2}, \dots, a_{s_k}$ that appear exactly once, or in other words, the count of values that are not repeated.

If your program makes more than 66 queries for one test case, or makes an invalid query, then the response to the query will be -1 . After receiving such a response, your program should immediately terminate to receive the verdict **Wrong Answer**. Otherwise, it may receive any other verdict.

When you determine the answer for the current test case, print

- $! x y z$

where x, y, z are the distinct three indices of the value that appears three times in a . You can print indices in any order.

Note that this output does not count towards the query limit of 66.

After that, proceed to the next test case or terminate your program if there are no more test cases.

After printing each query, do not forget to output the end of line and flush* the output. Otherwise, you will get the **Idleness limit exceeded** verdict.

If, at any interaction step, you read -1 instead of valid data, your solution must exit immediately. This means that your solution will receive **Wrong Answer** because of an invalid query or any other mistake. Failing to exit can result in an arbitrary verdict because your solution will continue to read from a closed stream.

Note that the interactor is **not adaptive**, which means that a is unchanged throughout the interaction.

Hacks

To make a hack, use the following format.

The first line should contain a single integer t ($1 \leq t \leq 500$), the number of test cases.

The first line of each test case should contain n ($2 \leq n \leq 1000$).

The second line of each test case should contain an array of length $2 \cdot n + 1$, the array a , which should fulfill the constraints of the problem.

The sum of n over all test cases should not exceed $2 \cdot 10^4$.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^4$.

Interaction

For each test case, first read a single integer n .

You may ask up to 33 queries in each test case.

To ask a query, print a line in the format:

- `? k s1 s2 ... sk ( $s_i \neq s_j$  for  $i \neq j$ ,  $1 \leq s_i \leq 2 \cdot n + 1$ )`

As a response to the query, you will receive the number of different values among $a_{s_1}, a_{s_2}, \dots, a_{s_k}$ that appear exactly once, or in other words, the count of values that are not repeated.

If your program makes more than 33 queries for one test case, or makes an invalid query, then the response to the query will be `-1`. After receiving such a response, your program should immediately terminate to receive the verdict `Wrong Answer`. Otherwise, it may receive any other verdict.

When you determine the answer for the current test case, print

- `! x y z`

where x, y, z are the distinct three indices of the value that appears three times in a . You can print indices in any order.

Note that this output does not count towards the query limit of 33.

After that, proceed to the next test case or terminate your program if there are no more test cases.

After printing each query, do not forget to output the end of line and flush* the output. Otherwise, you will get the `Idleness limit exceeded` verdict.

If, at any interaction step, you read `-1` instead of valid data, your solution must exit immediately. This means that your solution will receive `Wrong Answer` because of an invalid query or any other mistake. Failing to exit can result in an arbitrary verdict because your solution will continue to read from a closed stream.

Note that the interactor is **not adaptive**, which means that a is unchanged throughout the interaction.

Hacks

To make a hack, use the following format.

The first line should contain a single integer t ($1 \leq t \leq 500$), the number of test cases.

The first line of each test case should contain n ($2 \leq n \leq 1000$).

The second line of each test case should contain an array of length $2 \cdot n + 1$, the array a , which should fulfill the constraints of the problem.

The sum of n over all test cases should not exceed $2 \cdot 10^4$.

input
1
2
0
2
2
0
1

input
1
2
0
2
2
0
1
output
? 2 1 2
? 2 1 4
? 2 1 5
? 5 1 2 3 4 5
? 4 1 2 3 4
! 1 2 3

The secret array is $a = [1, 1, 1, 2, 2]$.

In the first query, we ask for the number of values that appear exactly once in $[a_1, a_2] = [1, 1]$, since value 1 is repeated, the answer is 0.

In the second query, we ask for the number of values that appear exactly once in $[a_1, a_4] = [1, 2]$, since values 1 and 2 appear exactly once, the answer is 2.

In the fourth query, we ask for the number of values that appear exactly once in $[a_1, a_2, a_3, a_4, a_5] = [1, 1, 1, 2, 2]$, since values 1 and 2 are repeated, the answer is 0.

At the end, we print that the indices of the value repeated three times are 1, 2 and 3.

D2. Unique Values (Hard version)

2 seconds, 256 megabytes

The difference between the easy version and hard version is the maximum number of queries allowed. In this version, it is 33.

There is a secret array a of length $2n + 1$, whose elements are integers from 1 to n . Each value appears exactly twice, except for one value, which appears exactly three times.

Your goal is to find the three positions of the value that appears three times.

To do this, you may ask the following query at most 33 times:

1. Choose an integer k and an array s of k distinct indices between 1 and $2n + 1$.
2. You will receive the number of different values among $a_{s_1}, a_{s_2}, \dots, a_{s_k}$ that appear exactly once, or, in other words, the count of values that are not repeated.

For example, if the values $a_{s_1}, \dots, a_{s_k}$ are $\{2, 1, 2, 3, 2, 3, 6, 7\}$, the answer to the query will be 3, as 1, 6, 7 are the only values that appear exactly once. 3 appears 2 times and 2 appears 3 times, which is more than once, so they are not counted.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 500$). The description of the test cases follows.

The first line of each test case contains a single integer n ($2 \leq n \leq 1000$).

The array a is fixed for the test case and does not change during the interaction. In other words, the interactor is **not adaptive**.

output

```
? 2 1 2
? 2 1 4
? 2 1 5
? 5 1 2 3 4 5
? 4 1 2 3 4
! 1 2 3
```

The secret array is $a = [1, 1, 1, 2, 2]$.

In the first query, we ask for the number of values that appear exactly once in $[a_1, a_2] = [1, 1]$, since value 1 is repeated, the answer is 0.

In the second query, we ask for the number of values that appear exactly once in $[a_1, a_4] = [1, 2]$, since values 1 and 2 appear exactly once, the answer is 2.

In the fourth query, we ask for the number of values that appear exactly once in $[a_1, a_2, a_3, a_4, a_5] = [1, 1, 1, 2, 2]$, since values 1 and 2 are repeated, the answer is 0.

At the end, we print that the indices of the value repeated three times are 1, 2 and 3.

E. Coloring a Red Black Tree

2 seconds, 256 megabytes

You are given a tree of n nodes numbered from 1 to n and a binary string s of length n where $s_i = 1$ means that the i -th node is colored red and $s_i = 0$ means that the i -th node is colored black. You are allowed to perform the following operation:

- Select a node a ($1 \leq a \leq n$).
- A neighbor b of a is selected uniformly at random.
- Recolor node a with the color of b . **Note that this changes the color of the vertex you selected, not the one that was uniformly chosen.**

You want to color all the nodes in the tree red by performing the operation any number of times (possibly zero). Please compute the minimum expected number of operations needed to do so if you perform them optimally. There will be at least one red node initially.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 1000$). The description of the test cases follows.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$).

The second line contains a binary string s of size n .

The following $n - 1$ lines each contain two integers, the i -th of them contains u_i and v_i ($1 \leq u_i, v_i \leq n$), which means that there is an edge between nodes u_i and v_i .

It is guaranteed that the given graph is a tree, there is at least one red node, and the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single real number representing the minimum expected number of operations.

Your answer is considered correct if its absolute or relative error does not exceed 10^{-6} .

Formally, let your answer be x , and the jury's answer be y . Your answer is accepted if and only if $\frac{|x-y|}{\max(1, |y|)} \leq 10^{-6}$.

input

```
3
3
101
1 2
2 3
5
10010
1 2
2 3
3 4
2 5
3
010
1 2
2 3
```

output

```
1.00000000000000000000
4.50000000000000000000
2.00000000000000000000
```

In the first test case, there is only one node colored in black, which has only two red colored neighbors, thus, if we first apply the operation to the second node, we will have a red colored tree in just one operation.

F. MEX Replacement on Tree

3.5 seconds, 256 megabytes

You are given a tree rooted at 1 with n vertices and a permutation p of length n consisting of integers $0, 1, \dots, n-1$, where p_v represents the weight of vertex v .

Let S_v denote the set of weights written on the vertices belonging to the path from 1 to v . Define a function f as $f(v) = \text{MEX}(S_v)$, that is, the MEX over the set consisting of the weights of vertices on the simple path between the root and v (inclusive), where MEX of a collection of integers $c_1, c_2, \dots, c_k$ is defined as the smallest non-negative integer x which does not occur in the collection c .

Also, given the following operation:

- Choose an integer v ($1 \leq v \leq n$) and assign $f(v)$ to p_v .

Your goal is to find the maximum value of $\sum_{v=1}^n f(v)$ after performing the above operation **no more than once** (possibly zero).

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 2 \cdot 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of vertices of the tree.

The second line of each test case contains n integers $p_1, p_2, \dots, p_n$ ($0 \leq p_i \leq n-1$) — the elements of array p . It is guaranteed that every integer from 0 to $n-1$ appears exactly once in p .

Then $n-1$ lines follow, each line contains two integers x_i and y_i ($1 \leq x_i, y_i \leq n$), indicating an edge between vertices x_i and y_i . It is guaranteed that these edges form a tree.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single line containing an integer: the maximum value of $\sum_{v=1}^n f(v)$ after performing the above operation no more than once (possibly zero).

input

```

7
1
0
3
1 0 2
1 2
1 3
6
1 4 5 2 0 3
1 4
1 5
6 2
2 5
2 3
5
1 2 3 0 4
1 2
2 3
3 4
4 5
10
9 8 7 1 3 2 5 4 6 0
6 10
3 1
8 7
4 2
2 8
7 5
10 9
3 9
6 4
9
8 4 0 6 5 7 3 1 2
8 3
4 2
4 6
9 3
7 6
1 5
8 5
9 2
7
1 5 2 3 6 0 4
7 6
4 3
7 2
5 1
2 4
6 5

```

output

```

1
4
12
9
30
26
17

```

4. Doing operation on vertex 3 would assign $f(3) = 0$ to p_3 , change $f(3)$ to 2, which leads to $\sum_{v=1}^n f(v) = 4$

So the answer is 4, by doing operation on vertex 3.

In the first example, not doing any operation is optimal, so the answer is 1.

In the second example, we have

- $f(1) = \text{MEX}(\{p_1\}) = \text{MEX}(\{1\}) = 0$
- $f(2) = \text{MEX}(\{p_1, p_2\}) = \text{MEX}(\{1, 0\}) = 2$
- $f(3) = \text{MEX}(\{p_1, p_3\}) = \text{MEX}(\{1, 2\}) = 0$

And we have the following options:

1. Doing nothing would lead to $\sum_{v=1}^n f(v) = 2$
2. Doing operation on vertex 1 would assign $f(1) = 0$ to p_1 , change $f(1), f(2), f(3)$ to 1, which leads to $\sum_{v=1}^n f(v) = 3$
3. Doing operation on vertex 2 would assign $f(2) = 2$ to p_2 , change $f(2)$ to 0, which leads to $\sum_{v=1}^n f(v) = 0$

[Codeforces](#) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform